Fast vs. Slow Hashing

When Speed Helps and When It Hurts

Why Choosing the Right Hash Matters

173,000 k/s

RC4 key recovery speed
(deprecated cipher)

7,400,000 k/s

SHA-512 hashing speed
(fast hash, same GPU)

~2 k/s

Argon2id cracking speed
(intentionally slow hash)

Using the wrong hash for the job wastes performance or compromises security.

Source: Hashcat Benchmarks (RTX 4090), OnlineHashCrack 2025

Two Categories of Hash Functions

Fast Hashes

Designed for speed and efficiency.
Optimized for high throughput.

SHA-256, SHA-3, SHA-512, BLAKE3

Billions of hashes per second on GPUs

Deterministic, fixed-length output

No tunable cost parameter

Slow Hashes

Intentionally expensive to compute.
Designed to resist brute-force attacks.

Argon2id, bcrypt, scrypt, PBKDF2

Tunable work factor (cost, memory, time)

Memory-hard variants resist GPU attacks

Target ~250ms per hash for authentication

When to Use Fast Hashes

File Integrity

Verify downloads and detect corruption. BLAKE3 hashes a 6.6 GB file in under 2 seconds.

Digital Signatures

Hash documents before signing with RSA or ECDSA. Speed enables bulk verification.

TLS / HTTPS

HMAC-SHA256 authenticates every packet in transit. Must run at wire speed.

Data Deduplication

Identify duplicate blocks in storage. Throughput is the bottleneck, not secrecy.

API Authentication

HMAC-based tokens (JWT, webhook signatures) need fast verification per request.

Checksums & Caching

Content-addressable storage, ETags, cache keys. xxHash also works here.

When to Use Slow Hashes

Any time a low-entropy secret (password, PIN, passphrase) must be stored or verified, use a slow, memory-hard hash.

Password Storage

User login credentials in any application. The primary use case for slow hashing.

Key Derivation

Deriving encryption keys from passphrases (e.g., disk encryption, wallet keys).

Token / PIN Storage

Short codes, backup codes, and API secrets derived from human input.

The goal: make each guess cost ~250ms. An attacker trying 10 billion passwords at 250ms each needs ~79 years.

Slow Hash Preference Order

1	**Argon2id**	Memory-hard | GPU-resistant | Side-channel safe	OWASP's top recommendation. Combines memory-hardness with hybrid resistance to both GPU brute-force and side-channel attacks. Tunable memory, iterations, and parallelism.
2	**scrypt**	Memory-hard | GPU-resistant	Strong GPU resistance via large memory requirement. More complex to tune (N, r, p) — weak parameters silently degrade security. No side-channel protection.
3	**bcrypt**	CPU-hard | 4 KB memory only	Battle-tested since 1999 with universal library support. Single cost parameter is easy to tune. Only 4 KB RAM — increasingly vulnerable to modern GPUs. 72-byte input limit.
4	**PBKDF2**	CPU-hard | No memory-hardness	Use only when FIPS-140 compliance is required. Highly GPU-parallelizable — attackers gain 10-30x advantage over CPU defenders. Requires 600,000+ iterations (OWASP minimum).

Source: OWASP Password Storage Cheat Sheet

What Memory-Hard and CPU-Hard Mean

Memory-Hard

Argon2id, scrypt

- Requires large RAM per hash (16–64 MiB or more)
- GPUs have limited memory per core — only hundreds of parallel threads, not millions
- Doubling memory quadruples attacker cost
- Memory bandwidth becomes the bottleneck, not raw compute

CPU-Hard

bcrypt, PBKDF2

- Requires many CPU cycles per hash but very little RAM (4 KB for bcrypt)
- Slows each core, but GPUs have thousands of cores to parallelize the work
- Attackers gain 10–30x speed on GPU hardware
- Effective when tuned aggressively, but diminishing returns vs. GPUs

Neither

SHA-256, SHA-512, MD5

- Designed for speed — no deliberate cost mechanism
- GPUs process billions of hashes per second with trivial memory
- No tunable work factor to increase cost over time
- Never use for passwords — one GPU can exhaust common password lists in minutes

Memory-hardness is the key differentiator. It turns the attacker's memory into the bottleneck — not just CPU cycles.

Secure Memory Wiping by Language

Language	Can Wipe?	Mechanism	Caveats
Rust	Yes	zeroize / secrecy crates; guaranteed on Drop	Best-in-class; compiler cannot optimize away
C / C++	Yes	memset_s, SecureZeroMemory, explicit overwrite	Plain memset can be optimized away by compiler
Go	Partial	memguard library; mlock + mmap syscalls	GC may copy data; must bypass runtime allocator
C#	Partial	CryptographicOperations.ZeroMemory, Span<T>	SecureString deprecated; GC moves objects in heap
Java	Partial	char[] + Arrays.fill(); PBEKeySpec.clearPassword()	Strings immutable; frameworks convert char[] to String
Swift	Partial	Mutable Data type; manual zeroing possible	ARC may leave copies; no built-in secure wipe
Python	No	bytearray possible but fragile; ctypes hacks	Strings immutable; GC non-deterministic; no control
JavaScript	No	Buffer.fill(0) for Buffers only	Strings immutable; V8 GC non-deterministic
Ruby	No	Strings mutable but no pinning API	GC copies freely; no memory layout guarantees

Prefer languages with explicit, guaranteed memory wiping for any code that handles passwords, keys, or tokens.

Source: Praetorian, Rust zeroize docs, Go memguard

Common Mistakes

Fast Hash for Passwords	Using SHA-256 or SHA-512 to store passwords. GPUs crack hundreds of millions of these per second.
Slow Hash for Integrity	Using bcrypt to checksum files. Wastes CPU and degrades throughput for no security benefit.
Unsalted Hashes	Even a slow hash without a unique salt is vulnerable to precomputed rainbow tables.
Static Work Factors	Failing to increase cost parameters as hardware improves. What was slow in 2015 is fast today.

Match the hash to the threat model. Speed is a feature for integrity; it is a vulnerability for passwords.

KEY TAKEAWAYS

Right Hash. Right Job.

- Fast hashes (SHA-256, BLAKE3) for integrity, signatures, and authentication tokens
- Slow hashes (Argon2id, bcrypt) for passwords, PINs, and passphrase-derived keys
- Never use a fast hash to store passwords — GPUs make it trivial to crack
- Never use a slow hash for bulk data — it wastes resources with no security gain
- Tune work factors to target ~250ms and increase them over time

Source: OWASP, NIST SP 800-63B